

OpoFit — Memoria resumida para el tribunal

Autor: Rafael Javier Luengo Barreno

Trabajo Fin de Grado

1. Por qué este proyecto

Cuando planteé el TFG, quería que no fuera solo “una app más”, sino algo con sentido para gente que conozco: opositores que compaginan estudio con preparación física y que, muchas veces, acaban entrenando a ciegas o copiando rutinas genéricas de internet que no tienen nada que ver con las pruebas reales de su convocatoria. Esa frustración —no saber si vas bien encaminado, no tener las marcas ordenadas, no ver el vínculo entre lo que haces en el gimnasio y lo que te van a exigir el día del examen— es la que intenté abordar con **OpoFit**.

La idea central es sencilla de explicar en una frase: **una aplicación móvil que acompaña al opositor en la parte deportiva**, conectada a un servidor que centraliza datos, reglas y contenidos, de modo que la experiencia sea coherente y personalizada según la oposición elegida.

2. Qué problema concreto resuelve

Antes de entrar en tecnología, prefiero situarnos en el contexto humano. Un opositor con pruebas físicas no solo “va al gimnasio”: compagina horas de estudio teórico con fatiga muscular, con dudas sobre si está priorizando bien y con la presión de una fecha de examen. Ese contexto me pareció el hilo conductor del proyecto: **reducir fricción** entre la información oficial y el día a día del entrenamiento.

Las oposiciones con pruebas físicas suelen publicar **baremos, requisitos mínimos y descripciones de pruebas**, pero esa información está dispersa en PDFs, webs institucionales y foros. El aspirante necesita, al mismo tiempo:

- Entender **qué debe rendir** y en qué unidades se mide cada prueba.
- Tener **rutinas de entrenamiento** que no contradigan lo que busca la convocatoria (por ejemplo, priorizar resistencia cuando su punto débil es la fuerza).
- **Registrar** lo que hace y **ver evolución**, no solo un historial vacío de fechas.

OpoFit intenta reunir esas necesidades en un solo flujo: el usuario se registra, elige oposición, consulta información contextual, entrena con rutinas alineadas a su perfil y va dejando constancia de sesiones y resultados por ejercicio. No sustituye al entrenador ni al criterio médico, pero sí **ordena la información y reduce la fricción** entre “quiero prepararme” y “tengo un plan y un registro”.

3. Objetivos que me marqué

Objetivo general: diseñar e implementar un sistema software compuesto por una API REST y una aplicación Android nativa que permita a opositores gestionar su preparación física vinculada a una oposición concreta.

Objetivos que fui concretando durante el desarrollo:

- Persistir usuarios, rutinas, sesiones y marcas en una **base de datos relacional** bien modelada, con integridad referencial.

- Ofrecer **autenticación** fiable (contraseña con hash, sesión con token JWT) y, además, integrar **inicio de sesión con Google y Firebase** para no cerrar la puerta a quien ya usa esas cuentas en el móvil.
 - Separar claramente **rutinas oficiales** (predefinidas según nivel, género y enfoque) de **rutinas libres** creadas por el propio usuario a partir de un catálogo de ejercicios.
 - Calcular de forma razonable un **nivel sugerido** a partir de las marcas del usuario y los baremos cargados en base de datos, sin dar una falsa precisión si faltan datos.
 - Mostrar **noticias y contexto** relacionados con la oposición mediante agregación de **feeds RSS** en el servidor, para que la app no dependa solo de datos estáticos.
 - Construir una interfaz en **Jetpack Compose** navegable, con pantallas de perfil, rutinas, entrenamiento, historial y ajustes.
-

4. Alcance y límites (honestos)

Un TFG vive en el tiempo: semanas de curso, otras asignaturas, y la necesidad de cerrar un entregable que se pueda evaluar. Por eso fijé un **perímetro** claro: lo que entra es lo que puedo mantener coherente de principio a fin; lo que queda fuera es trabajo futuro o mejoras que no son imprescindibles para demostrar competencias.

He sido consciente de que un TFG no puede cubrir todo lo que tendría un producto comercial. En mi alcance **no** entra, por ejemplo, un panel web de administración para que un funcionario cargue convocatorias al día; la carga de datos relevante se hace vía scripts SQL y semillas. Tampoco he montado un sistema de pagos, ni chat con entrenadores, ni notificaciones push complejas para recordatorios de entreno —son mejoras naturales para un trabajo futuro.

Sí he cuidado que lo implementado sea **coherente de punta a punta**: que un usuario real pueda registrarse, navegar, entrenar y ver su progreso sin saltos lógicos graves.

5. Arquitectura de la solución (visión de conjunto)

Opté por un modelo **cliente–servidor clásico**:

- **Cliente**: aplicación Android en Kotlin, con interfaz declarativa (**Compose**), navegación centralizada y capa de red con **Retrofit** que habla en JSON con el backend.
- **Servidor**: **Node.js** con **Express**, organizado en rutas, controladores y servicios que encapsulan la lógica y el acceso a **MySQL** mediante un pool de conexiones.

Esa separación me permitió razonar por capas: la app no “conoce” el SQL; solo conoce contratos HTTP. El servidor no “conoce” la UI; solo valida identidad, aplica reglas y devuelve datos. Cuando tuve que depurar un fallo, casi siempre sabía en qué lado empezar.

6. Base de datos: qué modelo de datos defendí

Cuando expliqué el esquema a amigos no técnicos, usé la metáfora de **cajones etiquetados**: un cajón de oposiciones, otro de pruebas que “pertenecen” a cada oposición, otro de usuarios que eligen una oposición, y así sucesivamente. Si mezclas etiquetas, el sistema puede guardar filas pero **pierde**

sentido en las pantallas. Por eso las claves foráneas y los índices únicos no son un capricho de quien disfruta normalizando: son la garantía de que lo que ves en la app corresponde a relaciones reales en el mundo del opositor.

El núcleo relacional gira en torno a **oposiciones** y **pruebas oficiales** (cada prueba enlazada a una oposición). Los **usuarios** tienen perfil físico (género, peso, altura, IMC calculado en el registro) y pueden asociarse a una oposición.

Las **rutinas oficiales** (`rutinas_opo`) se definen por nivel (básico, intermedio, avanzado), género, tipo de enfoque (fuerza, velocidad, resistencia) y oposición; el **detalle** de cada rutina son filas que enlazan ejercicios con series, repeticiones y descanso. Las **rutinas personalizadas** repiten la idea (cabecera + líneas) pero ligadas al usuario.

El **historial** guarda cada sesión con tipo (oficial o personal), duración y referencia a la rutina usada; los **resultados** por ejercicio van en una tabla aparte, lo que permite consultar evolución temporal por movimiento. Las **marcas de perfil** recogen la mejor marca conocida por prueba oficial para cruzarla con **baremos** y **requisitos por nivel**.

No entraré aquí en cada clave foránea —eso lo desarrollo en mi documentación técnica personal—, pero sí quiero dejar claro que **prioricé integridad** y **consultas claras** frente a tablas “todo en uno” difíciles de mantener.

7. El servidor: decisiones que tomé

En `app.js` configuré **CORS**, **JSON body parser**, **rate limiting** global y uno más estricto en rutas de autenticación para dificultar abuso de login. Las rutas cuelgan de prefijos como `/api/auth`, `/api/user`, `/api/oposiciones`, `/api/rutinas`, `/api/rutinas-pers`, `/api/historial`, `/api/info` y `/api/ejercicios`.

La **autenticación con JWT** la firmo en el servidor con un secreto en variable de entorno; las rutas privadas pasan por un **middleware** que comprueba el token en la cabecera `Authorization`. Los controladores se mantienen relativamente delgados: la parte fea de transacciones SQL y validaciones complejas la concentré en **servicios** (`AuthService`, `RutinasService`, `ProgresoService`, `RssService`, etc.).

Un ejemplo de regla de negocio que me importó dejar bien explicada: en **ProgresoService**, antes de insertar una sesión nueva, compruebo si **ya existe una sesión el mismo día** para la misma rutina y usuario —así evito duplicados por descuido y el mensaje de error es entendible (“ya registraste este entrenamiento hoy”).

Otro ejemplo: en **RutinasService**, el cálculo de **nota media y nivel sugerido** a partir de baremos solo tiene sentido cuando el usuario ha completado **todas** las pruebas de esa oposición en su perfil; si faltan marcas, devuelvo explícitamente que faltan datos en lugar de inventar un nivel con la mitad de la información.

8. La app Android: cómo organicé el front

Elegí **Kotlin** y **Jetpack Compose** porque encajan con el enfoque actual de Android y porque la UI como función del estado me ayudó a no perderme en callbacks. La navegación la concentré en un `NavHost` con destinos definidos como constantes; el arranque depende de si hay sesión iniciada (login o pantalla principal con barra inferior).

Cada área funcional tiene su **ViewModel** donde lanzo corrutinas, llamo a la API y expongo estado a la interfaz. Los modelos de petición y respuesta están separados en paquetes de `data`, de forma que cuando cambia un endpoint solo toco un sitio.

Para desarrollo, la URL base del API apunta al servidor en mi red local; el emulador y el móvil físico no se comportan igual con `localhost`, y eso fue una de las primeras “lecciones prácticas” que tuve que asumir con paciencia.

8.1. Cómo “hablan” la app y el servidor (sin entrar en código)

Aunque esta memoria no es el manual de API, sí quiero que el tribunal intuya el **contrato** entre cliente y servidor. Cada pantalla que muestra datos “vivos” corresponde a una o varias peticiones HTTP: algunas son públicas en el sentido de que no requieren sesión (por ejemplo listar oposiciones para el registro), y otras **exigen** cabecera de autorización. Ese patrón es el que permite que el mismo backend sirva a muchos usuarios sin mezclar datos: el token identifica al titular de la sesión y el servidor decide si puede leer o escribir en su nombre.

Cuando algo falla —red inestable, servidor apagado, token caducado— la app no debería quedarse en un spinner infinito. Preferí estados de error **legibles** y, cuando fue posible, mensajes que vengan del propio backend en español para no duplicar textos en dos sitios. No siempre se consigue el ideal; a veces el cliente muestra un mensaje genérico, pero la intención está ahí: **tratar el fallo como parte del flujo**, no como un accidente que el usuario no entiende.

9. Seguridad y privacidad (nivel TFG)

Las contraseñas **nunca** se guardan en claro: uso **bcrypt** con coste razonable. El JWT expira (en mi configuración, a las 24 horas), lo que obliga a renovar sesión de vez en cuando. Las rutas que devuelven datos de usuario exigen token válido.

Sobre **Google y Firebase**, el servidor no se fía del email que manda solo el cliente: verifica el **token de identidad** con las librerías correspondientes antes de crear o recuperar usuario. Eso fue un requisito que me impuse desde el principio: confiar en el proveedor de identidad, no en un campo JSON manipulable.

10. Funcionalidades que el tribunal puede relacionar con la pantalla

De forma resumida, el usuario puede:

- **Registrarse e iniciar sesión** (email/contraseña, Google, Firebase según flujo).
 - Ver y **editar perfil** y **ajustes** (unidades, etc.).
 - Consultar **información de su oposición**, requisitos por género y, donde aplica, **noticias vía RSS** agregadas en servidor.
 - Obtener su **entrenamiento del día** según la lógica de rutinas oficiales y su evolución de marcas.
 - **Crear, listar, ver en detalle y eliminar rutinas libres** montadas desde el catálogo de ejercicios.
 - **Registrar entrenamientos** con duración y resultados por ejercicio.
 - Revisar **historial de sesiones** y **evolución** por ejercicio.
 - Consultar **información de pruebas** y marcas en relación con baremos.
-

10 bis. Tres historias de uso (cómo lo imagino en la práctica)

Historia A — Primera semana. María se registra, elige oposición y rellena sus datos y marcas iniciales si las tiene. Explora la ficha de su convocatoria y entiende qué pruebas le esperan. Empieza a usar rutinas sugeridas y registra su primer entreno. Al cabo de unos días mira el historial: ya no depende de la memoria ni de capturas sueltas en el móvil.

Historia B — Punto intermedio. Luis lleva semanas usando la app. Crea una **rutina libre** para trabajar un déficit concreto (por ejemplo tracción) combinando ejercicios del catálogo. Alterna rutinas oficiales y libres; el historial distingue el tipo. Ve la **evolución** en un ejercicio concreto y ajusta esfuerzos la semana siguiente.

Historia C — Información viva. Ana revisa la sección de noticias agregada por RSS antes de una ronda de estudio. No espera magia: sabe que los feeds externos pueden fallar, pero cuando funcionan le dan contexto sin salir de la app.

Estas historias no son garantía de comportamiento en producción masiva; son la **brújula** con la que diseñé flujos y mensajes.

11. Cómo validé que “funciona”

Además de revisar la coherencia entre modelo de datos y pantallas, probé manualmente los flujos críticos: registro, login, cambio de perfil, creación de rutina libre, registro de entreno y consulta de historial. En el backend añadí pruebas unitarias puntuales donde tenía lógica sensible (por ejemplo el **middleware de token**), porque me daba más confianza al refactorizar.

No voy a presentar esto como una suite de pruebas exhaustiva al estilo empresa; sí como un **compromiso con la verificación** dentro del tiempo del TFG.

12. Dificultades y lo que aprendí

Hubo fricción real: alinear **emulador y red local**, entender **cuándo** el nivel de rutina se puede calcular sin engañar al usuario, y hacer tolerante el **RSS** cuando una fuente externa cambia de formato o va lenta. Cada uno de esos problemas me obligó a bajar del “tutorial perfecto” al **sistema que falla con mensaje claro**.

También aprendí a valorar la **documentación que escribo para mí mismo**: al cabo de semanas, sin notas, uno olvida por qué eligió una regla concreta; por eso mi documentación técnica completa — que no es esta memoria breve— va mucho más al detalle de implementación.

13. Conclusiones

Creo que OpoFit cumple el objetivo de **demostrar competencias** en diseño de software, bases de datos, API REST, seguridad básica y desarrollo Android moderno, con un **tema aplicado** que se puede defender oralmente sin recurrir solo a la teoría.

El proyecto es **ampliable**: notificaciones, panel de administración, tests automáticos más amplios, internacionalización o integración con wearables serían líneas naturales. Yo mismo lo dejo apuntado como trabajo futuro, no como excusa: aquí lo que cuenta es lo construido y **por qué** está construido así.

14. Agradecimientos

Agradezco la orientación académica recibida y el apoyo familiar y de amigos que me aguantaron las horas de depuración. Este trabajo es el resultado de ese equilibrio entre exigencia académica y ganas de hacer algo útil.

14 bis. Cómo evolucionó el proyecto en el tiempo (visión cronológica)

No escribí OpoFit en un solo bloque monolítico. Hubo una fase en la que solo existía base de datos y servidor, y la “app” era Postman o equivalente. Esa fase me pareció incómoda al principio —uno quiere ver botones— pero fue la que evitó que después tuviera que rehacer pantallas porque el modelo de datos era endeble. Luego llegó el login real y la sensación de “producto” empezó a aparecer. Después incorporé rutinas e historial, que es donde más se nota si las reglas de negocio están bien planteadas. El RSS y los detalles de información llegaron cuando ya podía confiar en que el núcleo no se caía al primer cambio. Ese orden no es el único posible, pero **sí el que a mí me funcionó** para no ahogarme en deuda técnica temprana.

15. Cómo organicé el trabajo (sin metodología “de manual”)

No adopté una metodología ágil formal con sprints y daily stand-ups porque el TFG es en gran medida un trabajo individual con hitos académicos. Lo que sí hice fue **ordenar el esfuerzo en fases** que me resultaban naturales:

Primero **modelé datos y servidor**, porque sin tablas coherentes la app solo puede mentir bien. Después **autenticación y primer flujo completo** (registro → token → llamada protegida), que es el cordón umbilical del resto. Luego **rutinas e historial**, que son el corazón deportivo del producto. El **RSS y la información de oposición** fueron una capa que aporta valor pero depende de que lo anterior sea sólido; la dejé para cuando ya podía confiar en el flujo principal.

Esa secuencia me salvó de un error típico: tener pantallas bonitas que no persisten nada útil. Preferí ver feo al principio pero **honesto en base de datos**.

16. Tabla de tecnologías (qué y por qué, en pocas líneas)

Capa	Tecnología	Por qué la elegí
Servidor	Node.js + Express	Ecosistema maduro, JSON natural, mucha documentación.
Datos	MySQL	Modelo relacional claro para integridad; cómodo de razonar en TFG.
Seguridad	bcrypt, JWT	Estándar de facto para contraseñas y sesiones stateless en API.
Auth social	Google OAuth, Firebase Admin	Lo que muchos usuarios ya usan en el móvil; verificación en servidor.
Cliente	Kotlin, Jetpack Compose	Línea actual de Android; menos boilerplate de UI que XML puro.
Red	Retrofit + corrutinas	Encaja con suspend y ViewModel; errores tratables sin crashear la UI.
RSS	rss-parser	Abstrae feeds heterogéneos; el servidor normaliza para el cliente.

No todas las elecciones son “las mejores del universo”; son **las que encajaban con mi curva de aprendizaje** y con entregar un sistema completo en tiempo razonable.

17. Descripción ampliada por bloques funcionales

Cuenta y perfil. El usuario debe identificarse de forma fiable. El registro clásico pide datos físicos porque el cálculo de IMC y el género entran en requisitos y baremos; no es capricho estético. Los ajustes de unidades permiten que la interfaz hable el idioma que el usuario espera (kilos, kilómetros, etc.) sin recalcular mal en silencio.

Oposición e información. La pantalla de oposición no es marketing: concentra **descripciones de pruebas, trucos** que ayudan a interpretar cada reto y enlaces a **requisitos** segmentados por género cuando la convocatoria lo distingue. El agregado de **RSS** intenta acercar noticias recientes sin que yo tenga que mantener un departamento de prensa: el servidor intenta leer feeds públicos y devolver una lista homogénea; cuando una fuente falla, el sistema puede degradarse con elegancia (lista vacía o mensaje) en lugar de tumbar toda la pantalla.

Rutinas oficiales. Aquí conecté la parte “inteligente” del backend con la experiencia de entreno. La rutina no es un PDF estático: depende de **nivel sugerido** derivado de marcas y baremos, y de **enfoque** (fuerza, velocidad, resistencia) que el usuario puede priorizar según su autoconocimiento. Eso implica consultas que cruzan varias tablas y reglas explícitas cuando los datos están incompletos.

Rutinas libres. Es el espacio donde el usuario **compone** su propio entrenamiento eligiendo ejercicios del catálogo. La complejidad está en mantener la coherencia entre cabecera de rutina, líneas de detalle y posterior registro en historial: todo debe referenciar los mismos identificadores.

Entrenamiento e historial. Registrar una sesión no es solo guardar una fecha: guardo **duración, tipo** de rutina y **resultados numéricos por ejercicio**, lo que permite graficar evolución. La regla de **no duplicar el mismo entreno el mismo día** para la misma rutina** es una decisión de producto: evita trampas accidentales y refuerza el hábito de “un entreno por día” para esa rutina concreta.

Progreso y marcas. Las marcas de perfil alimentan el cálculo de nivel y la lectura de baremos. Si el usuario mejora una marca, el sistema debería reflejarlo en siguientes recomendaciones; ese es el tipo de **feedback en cadena** que da sentido al proyecto a largo plazo.

18. Consideraciones éticas y de uso responsable

Una app de preparación física puede **incentivar** el entrenamiento, pero no sustituye valoración médica ni supervisión profesional. En la defensa oral soy consciente de que debo dejar claro que OpoFit es una **herramienta de organización y seguimiento**, no un dictamen médico ni un certificado de aptitud.

También traté los datos personales con el mínimo sensato: contraseñas protegidas, acceso a historial condicionado a identidad verificada, y sin exponer información de un usuario a otro mediante identificadores adivinables en la API.

19. Riesgos y limitaciones conocidas

Dependo de **servicios externos** (feeds RSS, proveedores OAuth) que pueden cambiar o caer.

Dependo de que el usuario **introduzca marcas con criterio**: si los datos de entrada son absurdos, la salida (nivel sugerido) también lo será. Y dependo de un **entorno de desarrollo** estable: cambios de versión en Gradle o en dependencias de Android pueden romper la compilación si no se documentan.

Ninguna de estas limitaciones invalida el TFG; **las reconozco** porque forma parte de un trabajo maduro saber dónde están los bordes.

20. Relación con otros enfoques (apps genéricas de gimnasio)

Las aplicaciones tipo “tracker de musculación” suelen ser excelentes para series y repeticiones, pero raramente entienden **baremos de oposición** ni **requisitos por convocatoria**. OpoFit no compite en profundidad social con redes fitness; compite en **especialización** para un nicho concreto. Esa fue mi apuesta: menos features genéricas, más **sentido** para quien estudia oposiciones con prueba física.

21. Qué enseñaría en una demo en vivo

Si tuviera quince minutos frente al tribunal, priorizaría: **login** → **pantalla principal** → **rutina o entreno** → **registro de sesión** → **historial o evolución**. Es el “camino feliz” que demuestra que las piezas encajan. Un segundo bloque sería **crear rutina libre** y **consultar información de oposición** con noticias. El resto —ajustes, borrado de cuenta, matices de Firebase— puede quedar como pregunta del tribunal.

22. Trabajo futuro que me gustaría abordar

Me gustaría un **panel web** para administrar oposiciones y baremos sin tocar SQL a mano, **tests de integración** automatizados sobre la API, y **notificaciones** discretas para recordar entrenamientos. También me interesaría explorar **sincronización offline** parcial en el móvil para gimnasios con mala cobertura, aunque eso ya implica conflictos de reconciliación que no abordé aquí.

23. Cierre personal

Este proyecto me ha hecho pasar por **todas las capas** de un sistema real: desde el diagrama entidad-relación hasta el píxel en Compose, pasando por el token que nunca debe viajar en claro. Si al leer esta memoria se entiende **qué problema hay, cómo lo resolví y con qué matices**, habré cumplido lo que pretendía con este texto para el tribunal.

24. Fuentes de información y convenciones que seguí

Para el contenido deportivo-administrativo (descripciones de pruebas, enfoques de entrenamiento) me apoyé en la documentación oficial de convocatorias, en guías generales de preparación física y en criterios de tutores académicos. La parte puramente técnica la contrasté con documentación oficial de Android Developers, Express, MySQL y las librerías que uso. Cuando una decisión era ambigua —por ejemplo, si permitir dos sesiones el mismo día para la misma rutina— opté por **reglas explícitas** reflejadas en código, no por un comportamiento que “a veces” funciona y otras no.

25. Entorno de desarrollo y reproducibilidad

Para que el proyecto no dependa solo de mi equipo, dejé en el repositorio los scripts de base de datos y un ejemplo de variables de entorno para el backend. Quien tenga Node, MySQL y Android Studio en versiones razonables puede levantar el servidor, cargar datos y enlazar la app con la IP adecuada. No es despliegue profesional en la nube, pero sí un **mínimo de transferibilidad** que me parecía honesto exigirme.

26. Reflexión sobre la experiencia de usuario

Intenté que los mensajes de error fueran entendibles (“ya registraste este entrenamiento hoy”) en lugar de códigos crípticos. Los flujos largos los pensé por pasos: identidad, datos físicos, oposición. La barra inferior de navegación sigue patrones habituales en Android; reduce la curva de aprendizaje sin manual interminable.

27. Síntesis para una lectura rápida

- **Problema:** preparar pruebas físicas de oposición con información y seguimiento ordenados.
 - **Solución:** API REST + MySQL + app Kotlin/Compose.
 - **Qué aporta:** especialización frente a apps fitness genéricas; reglas y datos centralizados.
 - **Límites:** sin panel de administración completo; alcance académico.
 - **Aprendizaje:** encajar identidad, datos y reglas sin contradicciones entre capas.
-

28. Cómo encajan las competencias del grado (en mi lectura)

Sin ponerme la medalla, sí quiero dejar explícito qué competencias intenté demostrar con OpoFit: ingeniería del software (capas y responsabilidades), bases de datos (modelo coherente e integridad), desarrollo de servicios (API REST y seguridad básica), desarrollo móvil (interfaz y navegación) y

capacidad de documentar y defender un trabajo autónomo. No pretendo cubrir todo el catálogo de competencias en un solo proyecto; sí dar **anclas** para situar el TFG en lo que suele esperarse de un trabajo de esta naturaleza.

29. Qué vería el tribunal si abriera la app (recorrido guiado)

La pantalla de acceso ya muestra la apuesta por identidad y formularios razonables. La pantalla principal con barra inferior organiza rutinas, perfil, historial y atajos hacia entrenamiento e información. La información de oposición mezcla datos persistidos con noticias dinámicas cuando el RSS responde. Las rutinas separan el camino guiado por el sistema del taller de rutinas libres. El registro de entreno y el historial cierran el ciclo: deporte y seguimiento. Es el **guión** que preparo para una defensa oral centrada en el producto, no solo en el código.

30. Alternativas técnicas que valoré y descarté (sin dogmas)

Otro lenguaje en servidor, otra base SQL, interfaz en XML en lugar de Compose... son caminos válidos. Mis decisiones buscaron **equilibrio** entre tiempo de TFG, curva de aprendizaje y resultados verificables. Lo importante no es que el tribunal esté de acuerdo en cada tecnología, sino que yo pueda **argumentar** por qué las elegí y qué habría implicado cambiar.

31. Deuda técnica que reconozco

Con más tiempo modularizaría algunas piezas, unificaría formatos de error en todas las rutas y quizá introduciría validación declarativa de cuerpos JSON. Lo digo sin dramatismo: reconocer deuda es parte de madurez en ingeniería. Lo que no aceptaría es confundir un TFG funcional con un producto enterprise cerrado al cien por cien.

32. Mensaje final

OpoFit es un sistema pensado para un usuario concreto, con reglas que yo escribí y límites que no oculto. Esta memoria resume la **intención** y el **alcance**; mi documentación técnica completa es donde bajo al detalle de implementación para poder reconstruir o ampliar el proyecto con rigor.

33. Preguntas que me preparo para la defensa (respuestas cortas)

¿Por qué MySQL? Modelo relacional claro, herramientas accesibles y despliegue local sencillo para quien evalúe o reproduzca el trabajo.

¿JWT en el móvil? Mecanismo habitual; el token expira y la contraseña no viaja en claro. Mitigaciones adicionales serían propias de un producto maduro.

¿Si falla el RSS? El núcleo deportivo sigue; lo dinámico se degrada sin tumbar la app entera.

¿Privacidad entre usuarios? Rutas protegidas y coherencia entre identidad y recursos solicitados.

¿Escalabilidad masiva? No es el objetivo del TFG; sí lo es una base razonable para evolucionar.

34. Accesibilidad y producto real

No audité la app con estándares WCAG exhaustivos: prioricé funcionalidad y coherencia dentro del tiempo del TFG. Lo digo sin excusas vacías: en un producto profesional, accesibilidad sería requisito, no añadido opcional.

35. Datos que cambian con la convocatoria

Los baremos y pruebas dependen de normativa externa. OpoFit centraliza datos cargados en base; actualizar convocatorias requeriría mantenimiento o un panel admin —línea natural de trabajo futuro.

36. Epílogo personal

Cerrar un TFG no es solo entregar archivos: es mirar atrás y ver si el recorrido tiene sentido. Yo me llevo la idea de que un proyecto software es una cadena de decisiones que hay que saber explicar, la paciencia aprendida en depuración y la humildad de pedir ayuda cuando me atascaba. Si el tribunal valora el esfuerzo y la coherencia tanto como el resultado puntual, habré encontrado el equilibrio que buscaba al empezar.

Documento orientado al tribunal. La descripción técnica exhaustiva del sistema, pensada para mi uso y para la defensa detallada, constituye un volumen aparte: la documentación completa del proyecto.